# FHIR API Data Ingestion & Analytics

## Medallion Lakehouse Architecture — Technical Project Document

---

Platform: Databricks Free Tier | Storage: Unity Catalog + Delta Lake
Source: HAPI FHIR R4 Public API | GitHub: gopinadhghanta999/FHIR_PGN

Author: Gopinadh Ghanta | April 2026

# 1. Project Overview

This project implements a full end-to-end data engineering pipeline that ingests healthcare data from the publicly available HAPI FHIR R4 API, processes it through a Medallion Lakehouse Architecture on Databricks, and produces analytics-ready Gold layer tables suitable for business reporting and clinical analytics.

The pipeline ingests four FHIR resource types — Patient, Encounter, Observation, and Condition — across 3 days using incremental pagination. Data flows through Raw, Bronze, Silver, and Gold layers with data quality enforcement and SCD Type 2 versioning applied at the Silver layer via Delta Lake MERGE SQL statements.

## 1.1 Objectives

- Fetch incremental data from HAPI FHIR public API for 3 days with pagination
- Store raw JSON responses as-is in the Raw layer bucketed by date
- Convert JSON to structured Delta tables in Bronze with metadata columns
- Clean, deduplicate, and apply SCD Type 2 using Delta MERGE SQL in Silver
- Build a Star Schema Gold layer with dimension, fact, and aggregation tables
- Orchestrate the full pipeline end-to-end using Databricks Workflows via YAML

## 1.2 Technology Stack

| Component | Technology |
|---|---|
| Cloud Platform | Databricks Free Tier (Serverless Notebooks) |
| Storage | Unity Catalog — fhir_data catalog, 4 schemas |
| Processing | Apache Spark (PySpark) + Spark SQL |
| File Format | JSON (Raw), Delta Lake (Bronze / Silver / Gold) |
| SCD2 Method | Delta Lake MERGE SQL (not row_hash — field-level comparison) |
| Gold Write Mode | Gold tables refreshed on every run |
| Orchestration | Databricks Workflows — YAML-based job definition |
| Source System | HAPI FHIR R4 — https://hapi.fhir.org/baseR4 |
| Version Control | GitHub — gopinadhghanta999/FHIR_PGN |

# 2. Architecture & Catalog Structure

## 2.1 Medallion Layers

| Layer | Format | Purpose |
|---|---|---|
| Raw | JSON files | Store full API responses as-is, bucketed by date partition — no transformation |
| Bronze | Delta Tables | Parse JSON, explode entries array, add metadata columns — append mode daily |
| Silver | Delta + SCD2 | Parse resource JSON, flatten schema per resource, apply SCD2 via MERGE SQL |
| Gold | Delta Tables | Star schema — 5 dims, 3 facts, 5 aggregations — overwrite mode on each run |

## 2.2 Unity Catalog Structure

The project uses a dedicated catalog named fhir_data with four schemas — one per layer. Raw data is stored in a Unity Catalog Volume (not a Delta table). Bronze, Silver, and Gold data are stored as managed Delta tables registered in the catalog.

```
fhir_data  (catalog)
├── prd_raw    (schema)  → Volume: firstv  →
/Volumes/fhir_data/prd_raw/firstv/raw/{resource}/date={date}/
├── prd_bronze (schema)  → Tables: patient, encounter, observation, condition
├── prd_silver (schema)  → Tables: patient, encounter, observation, condition  (+SCD2
columns)
└── prd_gold   (schema)  → Tables: dim_patient, dim_date, dim_encounter,
                                   dim_observation_type, dim_condition_type,
                                   fact_patient_encounter, fact_observation,
fact_condition,
                                   agg_patient_summary, agg_condition_prevalence,
                                   agg_observation_stats,
agg_encounter_by_age_gender,
                                   agg_monthly_activity
```

# 3. Pipeline Notebooks

| # | Notebook | Purpose |
|---|---|---|
| 1 | Architecture_configuration | Creates fhir_data catalog, 4 schemas, raw Volume, raw folder structure |
| 2 | 01_Raw_ingesting | Fetches 4 resources × 3 days from FHIR API with pagination, saves JSON to Volume |
| 3 | 02_Bronze_layer | Reads JSON, explodes entries, adds metadata, writes Delta tables to prd_bronze |
| 4 | silver_patient_processing | Parses Patient JSON, flattens schema, applies SCD2 MERGE into prd_silver.patient |
| 5 | silver_processing_encounter | Parses Encounter JSON, flattens, applies SCD2 MERGE into prd_silver.encounter |
| 6 | silver_processing_observation | Parses Observation JSON, flattens, applies SCD2 MERGE into prd_silver.observation |
| 7 | silver_processing_condition | Parses Condition JSON, flattens, applies SCD2 MERGE into prd_silver.condition |
| 8 | FHIR_Gold_Layer_StarSchema | Reads Silver current records, builds 13 Gold tables — overwrite mode |

## 3.1 Raw Layer — 01_Raw_ingesting

Fetches data from the HAPI FHIR public API for each resource across 3 consecutive days. Uses FHIR's date filter and pagination to collect incremental data. Each response is saved as a JSON file to the Unity Catalog Volume.

**API filter:** _lastUpdated=ge{date}T00:00:00&_lastUpdated=lt{next_date}T00:00:00

**Page size:** 100 records per page, up to 3 pages per resource per day

**Storage path:** /Volumes/fhir_data/prd_raw/firstv/raw/{resource}/date={date}/response_{date}.json

Each JSON file contains a metadata wrapper including resource_type, run_date, extraction_timestamp, api_call_timestamp, api_urls_called, total_records, and the full entries array from the FHIR bundle response.

## 3.2 Bronze Layer — 02_Bronze_layer

Reads each raw JSON file, detects the entries array structure, explodes it so each FHIR resource record becomes one row, serialises the nested resource struct to a JSON string, and adds metadata columns before writing to managed Delta tables in prd_bronze.

| Metadata Column | Description |
|---|---|
| extraction_timestamp | Timestamp when the Bronze write occurred |
| api_url | Base FHIR API URL for this resource |

| | |
|---|---|
| load_date | Date of this ingestion run (YYYY-MM-DD) |
| source_file | Metadata file path from Spark _metadata column |

The resource column in Bronze is stored as a JSON string (to_json applied during Bronze load). This is the correct design — Silver notebooks then parse this string back into a struct using from_json with explicit schemas per resource.

Bronze tables use append mode — all daily loads accumulate. Delta table history is verified after each load to confirm version tracking.

## 3.3 Silver Layer — Resource-Specific Notebooks

Each of the four Silver notebooks follows the same pattern: read from prd_bronze, parse the JSON resource string with an explicit schema, flatten nested/array fields into tabular columns, apply data quality filters, and write to prd_silver via SCD Type 2 MERGE SQL.

All four Silver notebooks run in parallel in the Databricks Workflow — they have no dependency on each other since each reads from and writes to a different table.

### Per-Resource Schema and Flattening

| Resource | Key Transformations |
|---|---|
| Patient | explode_outer on name array → extracts family_name and given_name. CASE expression cleans duplicate family name. Derives age (datediff / 365.25) and age_group (Minor / Young Adult / Middle Age / Pre-Senior / Senior). Regex extracts and validates phone numbers and email addresses from telecom array. |
| Encounter | explode_outer on identifier array. Splits subject.reference and serviceProvider.reference on '/' to extract patient_id and organization_id. Classifies identifier_type (NUMERIC / INPATIENT / UNKNOWN). Filters to valid statuses: finished, in-progress, planned, cancelled. Window dedup on encounter_id by last_updated_ts desc. |
| Observation | explode_outer on code.coding array. coalesce on valueQuantity.value and valueInteger for observation_value. coalesce on effectiveDateTime and effectivePeriod.start for effective_ts. Classifies observation_type and data_source_type. Cleans unit strings. |
| Condition | explode_outer on code.coding and clinicalStatus.coding arrays. Extracts condition_code, condition_display, coding_system, clinical_status. Parses onset_ts and recorded_date. Adds dq_flags column for data quality tracking. Filters null condition_id. |

## 3.4 SCD Type 2 — Implementation Details

SCD Type 2 in this project is implemented using Delta Lake MERGE SQL statements — not row hash comparison. Each Silver notebook creates a temporary view of the incoming processed data and merges it into the Silver Delta table using field-level comparison in the WHEN MATCHED condition.

### SCD2 Columns on Every Silver Table

| Column | Description |
|---|---|
| effective_start_date | Timestamp when this version became active — set to current_timestamp() on INSERT |
| effective_end_date | Timestamp when this version was superseded — NULL for current records, set on MATCH UPDATE |
| is_current | Boolean — True for the latest active version, False for expired historical versions |

### MERGE Logic Per Resource

The MERGE statement for each resource follows this two-clause pattern:

| MERGE Clause | Action |
|---|---|
| WHEN MATCHED AND (field changes) | UPDATE: set is_current = false, effective_end_date = current_timestamp() — expires the old version |
| WHEN NOT MATCHED | INSERT: insert full new row with is_current = true, effective_start_date = current_timestamp(), effective_end_date = NULL |

### Field-Level Change Detection Per Resource

| Resource | Fields Compared in WHEN MATCHED Condition |
|---|---|
| Patient | gender, birth_date, last_updated, source, resource_type, version_id, active, last_name, first_name, name_use, phone_number, email, age, age_group |
| Encounter | status, patient_id, organization_id, last_updated_ts, source, version_id, identifier_type, identifier_numeric — matched on encounter_id + is_current = true |
| Observation | patient_id, encounter_id, observation_type, data_source_type, code, code_text_clean, observation_value, unit_clean, unit_system_clean, status, effective_ts, last_updated_ts, version_id — uses <> (strict comparison) |
| Condition | resource_type, condition_code, condition_display, coding_system, condition_text, patient_id, encounter_id, clinical_status, onset_ts, recorded_date, last_updated_ts, source, version_id, dq_flags — uses <=> (null-safe comparison) |

Note: Patient and Condition use different null-handling strategies. Patient uses standard <> comparison. Condition uses the null-safe <=> operator (NOT col1 <=> col2) which correctly handles NULL vs NULL as equal — preventing false change detection when optional fields are absent.

### Important — Changed Record Re-Insert

The standard two-clause MERGE (MATCHED → expire, NOT MATCHED → insert) handles new records correctly. However for changed records, the MATCHED clause only expires the old version — it does not insert the new version in the same MERGE pass. The new version is inserted because after expiry the record no longer matches (is_current = false), so subsequent runs or the same run via a second MERGE pass will insert it as NOT MATCHED. This is a known pattern when using MERGE for SCD2 without a WHEN NOT MATCHED BY SOURCE clause.

# 4. Gold Layer — Star Schema

The Gold layer reads only the latest state of each Silver table and builds a fully analytical Star Schema. All 13 Gold tables are written using overwrite mode — they are always a complete, fresh snapshot reflecting the current state of the Silver data. Gold never accumulates history — that is the responsibility of Silver.

**Write mode:** overwrite with overwriteSchema = true on every pipeline run

**Source:** Silver tables — all records (Gold applies its own row_number deduplication)

**Deduplication in Gold:** row_number() over partitionBy(id).orderBy(last_updated desc) = 1 — picks the latest version per record

## 4.1 Dimension Tables

| Table | Key Columns |
|---|---|
| dim_patient | patient_key, patient_id, gender, birth_date, age, age_group, first_name, last_name, phone_number, email, is_active, source_system, last_updated_ts, dw_load_date |
| dim_date | date_key (yyyyMMdd int), full_date, year, quarter, month, month_name, day_of_month, day_name, week_of_year, quarter_label, is_weekend — spans 2015-01-01 to 2030-12-31 |
| dim_encounter | encounter_key, encounter_id, patient_id, encounter_status, resource_type, identifier_system, identifier_type, identifier_numeric, organization_id, source, last_updated_ts, dw_load_date |
| dim_observation_type | observation_type_key (dense_rank), observation_code, observation_name, observation_type, data_source_type, default_unit, unit_system |
| dim_condition_type | condition_type_key (dense_rank), condition_code, condition_display, coding_system, resource_type |

## 4.2 Fact Tables

| Table | Key Columns & Measures |
|---|---|
| fact_patient_encounter | encounter_key, patient_id, date_key (ingestion_date yyyyMMdd), encounter_status, patient_age_at_encounter, patient_age_group, patient_gender, is_finished (0/1 flag), encounter_count (lit 1), dw_load_date — partitioned by ingestion_date |
| fact_observation | observation_key (try_cast BIGINT), patient_id, encounter_key, observation_type_key, observation_code, observation_name, observation_type, data_source_type, observation_value (float), unit, status, effective_date_key, ingestion_date_key, effective_ts, observation_count (lit 1) — partitioned by dw_load_date |
| fact_condition | condition_key (try_cast BIGINT), patient_id, encounter_key, condition_type_key, condition_code, condition_display, coding_system, clinical_status, is_active_condition (0/1 flag), dq_flags, ingestion_date_key, condition_count (lit 1) — partitioned by dw_load_date |

## 4.3 Aggregation Tables

| Table | What It Contains |
|---|---|
| agg_patient_summary | Per-patient 360 view: total_encounters, finished_encounters, last_encounter_date, total_conditions, active_conditions, condition_list (collect_set), total_observations, avg_heart_rate_bpm (code 8867-4), avg_diastolic_bp (8462-4), avg_systolic_bp (8480-6), avg_body_weight_kg (29463-7), avg_body_height_cm (8302-2) |
| agg_condition_prevalence | Per condition: patient_count, total_diagnoses, prevalence_pct (patient_count / total_patients * 100) — ordered by patient_count desc |
| agg_observation_stats | Per observation type: reading_count, patient_count, avg_value, min_value, max_value, stddev_value, p25_value, median_value, p75_value — full statistical profile |
| agg_encounter_by_age_gender | Encounter volume by patient_age_group × patient_gender × encounter_status: encounter_count, unique_patients |
| agg_monthly_activity | Month-over-month trend (year_month format): encounter_count, unique_patients, observation_count — outer join of monthly encounter and observation aggregates |

# 5. Pipeline Orchestration — Databricks Workflow

The complete pipeline is orchestrated as a Databricks Workflow defined in YAML (FHIR_Main_Workflow.yml) stored in the GitHub repository. All notebooks are sourced directly from GitHub — Databricks pulls them from the main branch at runtime, ensuring the workflow always uses the latest committed code.

## 5.1 Workflow DAG

| Task Key | Depends On |
|---|---|
| Medallain_architecture_creation | — (starts pipeline) |
| Raw_Data_Ingestion | Medallain_architecture_creation |
| Bronze_ingesiton | Raw_Data_Ingestion |
| Silver_Patient | Bronze_ingesiton (runs in parallel with other silver tasks) |
| silver_encounter | Bronze_ingesiton (runs in parallel) |
| silver_observation | Bronze_ingesiton (runs in parallel) |
| silver_condition | Bronze_ingesiton (runs in parallel) |
| Loading_To_Gold | Silver_Patient + silver_encounter + silver_observation + silver_condition (all 4 must succeed) |

## 5.2 Workflow Settings

| Setting | Value |
|---|---|
| Job Name | FHIR_Data_processing |
| Notebook Source | GIT — notebooks pulled from GitHub at runtime |
| Git Repository | https://github.com/gopinadhghanta999/FHIR_PGN.git |
| Git Branch | main |
| Failure Email | ghantagopinadh999@gmail.com |
| Queue | Enabled — prevents overlapping concurrent runs |
| Performance | PERFORMANCE_OPTIMIZED |
| Compute | Serverless Notebook Compute (Databricks free tier) |

# 6. Gold Layer Data Model — Table Relationships

The Gold layer follows a standard Star Schema. Fact tables join to dimension tables via surrogate or natural keys. The dim_date table uses integer date keys in yyyyMMdd format for efficient partitioned joins.

| Fact Table | Dimension Table | Join Key |
|---|---|---|
| fact_patient_encounter | dim_patient | patient_id |
| fact_patient_encounter | dim_encounter | encounter_key |
| fact_patient_encounter | dim_date | date_key → ingestion_date yyyyMMdd |
| fact_observation | dim_patient | patient_id |
| fact_observation | dim_observation_type | observation_type_key |
| fact_observation | dim_date | effective_date_key → effective_ts yyyyMMdd |
| fact_condition | dim_patient | patient_id |
| fact_condition | dim_condition_type | condition_type_key |
| fact_condition | dim_date | ingestion_date_key → ingestion_date yyyyMMdd |

# 7. Key Design Decisions

## 7.1 SCD2 via MERGE SQL — Not Row Hash

SCD Type 2 is implemented using Delta Lake MERGE SQL with explicit field-level comparison in the WHEN MATCHED clause, rather than computing a row hash. This approach was chosen for clarity and explicitness — the exact fields being tracked for changes are visible directly in the MERGE statement, making the versioning logic easy to audit and extend. The trade-off is that the MERGE condition is longer, but each field comparison is deterministic and requires no additional hash column to maintain.

## 7.2 Gold Layer Uses Overwrite Mode

Gold tables are written with mode overwrite on every pipeline run. This is intentional — Gold always reflects the current state of Silver data. Since Silver already maintains the full version history via SCD2, there is no value in accumulating Gold records across runs. Overwrite keeps Gold tables clean, query-efficient, and free of duplicate rows. If incremental Gold were needed, the correct approach would be to add a date partition and do partition-level overwrite — but for this project full overwrite is sufficient.

## 7.3 Gold Reads All Silver Records — Not Just is_current

The Gold layer reads all Silver records and applies its own row_number() deduplication (row_number over partitionBy(id) orderBy last_updated desc = 1) rather than filtering on is_current = true. This ensures Gold always picks the most recent version even if is_current has not yet been updated due to a partial pipeline run. It is a more defensive approach than relying solely on the is_current flag.

## 7.4 Explicit StructType Schemas in Silver

All Silver notebooks define explicit PySpark StructType schemas for from_json parsing rather than using schema inference. This guarantees deterministic column types, prevents schema drift across daily runs, and significantly improves Spark performance by avoiding the schema inference scan. Each resource has its own schema definition matching the actual FHIR R4 structure for that resource type.

## 7.5 resource Column Serialised to JSON String in Bronze

During Bronze load, the nested resource struct is serialised to a JSON string using to_json(). This is deliberate — it preserves the raw FHIR structure in Bronze without schema enforcement, making Bronze a faithful staging area. Silver notebooks then parse this string back into structs using from_json with the explicit per-resource schemas. This two-stage approach decouples raw storage from structured parsing.

## 7.6 Silver Notebooks Run in Parallel

The four Silver notebooks are independent of each other — each reads from its own Bronze table and writes to its own Silver table. The Databricks Workflow exploits this by running all four as parallel tasks after Bronze completes. The Gold task waits for all four to succeed before running.

# 8. How to Run the Pipeline

## Option A — Via Databricks Workflow (Recommended)

1. Open Databricks workspace → Workflows → Create Job
2. Click the three-dot menu → Edit as YAML
3. Paste the contents of FHIR_Main_Workflow.yml from the GitHub repository
4. Ensure the Git source is connected to your GitHub account
5. Click Save → Run Now
6. Monitor each task in the Runs tab — Silver tasks run in parallel after Bronze

## Option B — Run Notebooks Manually (In Order)

7. Architecture_configuration — creates catalog, schemas, volume, folder structure
8. 01_Raw_ingesting — fetches 3 days of data for all 4 resources
9. 02_Bronze_layer — converts JSON to Delta, adds metadata
10. silver_patient_processing — Silver SCD2 for Patient
11. silver_processing_encounter — Silver SCD2 for Encounter
12. silver_processing_observation — Silver SCD2 for Observation
13. silver_processing_condition — Silver SCD2 for Condition
14. FHIR_Gold_Layer_StarSchema — builds all 13 Gold tables

## Prerequisites

- Databricks Free Tier workspace with Serverless Notebook Compute
- GitHub repository connected to Databricks as a Git provider in workspace settings
- No external authentication — HAPI FHIR API is fully public
- All Python libraries used (pyspark, delta, requests, datetime, logging) are included in Databricks runtime

# 9. GitHub Repository Structure

**Repository:** https://github.com/gopinadhghanta999/FHIR_PGN.git
**Branch:** main

```
FHIR_PGN/
├── Architecture_configuration.ipynb
├── 01_Raw_ingesting.ipynb
├── 02_Bronze_layer.ipynb
├── silver_patient_processing.ipynb
├── silver_processing_encounter.ipynb
├── silver_processing_observation.ipynb
├── silver_processing_condition.ipynb
```

```
├── FHIR_Gold_Layer_StarSchema.ipynb
└── FHIR_Main_Workflow.yml
```

## 10. Submission Checklist

| | Requirement |
|---|---|
| ✅ | Incremental ingestion for 2–3 days with pagination (3 days, up to 3 pages per resource per day) |
| ✅ | Medallion Architecture — Raw → Bronze → Silver → Gold implemented across 4 schemas |
| ✅ | Raw layer stores API responses as-is in JSON files bucketed by date partition |
| ✅ | Bronze layer converts JSON to Delta tables with metadata — extraction_timestamp, api_url, load_date, source_file |
| ✅ | SCD Type 2 implemented in all 4 Silver tables via Delta MERGE SQL — is_current, effective_start_date, effective_end_date |
| ✅ | Silver layer cleans, deduplicates, and flattens nested FHIR JSON per resource with explicit schemas |
| ✅ | Gold layer Star Schema — 5 dimension tables, 3 fact tables, 5 aggregation tables (13 total) |
| ✅ | API calls orchestrated in order: Patient → Encounter → Observation → Condition |
| ✅ | Modular code — CONFIG dict, no hardcoded paths or dates in production flows |
| ✅ | Databricks Workflow YAML for full pipeline orchestration with email failure notification |
| ✅ | All notebooks and workflow YAML committed to GitHub repository |
| ✅ | Documentation of architecture, pipeline, table relationships, and design decisions |